Group 3 - Space Complexity

Members:
Kristy Mok, Ben Chalk, Corina Rivero Montiel, Euan Faller, Joel Cutler, Luke Callen, Sam Ade Fowodu

Change Report

a)

For our group to streamline the process of inheriting our chosen group's project, we initially discussed how we were going to clearly identify and track our changes and updates made to the original deliverables. We collectively decided on the following processes, tools and conventions below:

Deliverables management and version tracking:

- Google Drive – Fig. 1 shows how it is used as a main shared repository to organise and save files from each project section, allowing for efficient and reliable accessibility. We have a folder for the updated deliverables, where we made copies of the original documents to build upon.
- Google Docs – utilised for collaborative editing and to review changes made via Version History as shown in Fig. 2, to oversee the changes made to the original documents we inherited. In addition, comments illustrated in Fig. 3 allowed members to provide contextual feedback and propose edits. Overall, this ensured transparency in modifications and alignment.

Change and update tracking identifier:

- Colour coding – by highlighting additions made to the original documents in green and orange for modifications as shown in Fig [2], it provided a visual system for us to easily distinguish our work from the original.

Team planning and status updates:

- Minutes in Google Docs – this was a shared document for team members to reference back to the key discussion points and latest schedule updates from our weekly meetings. This held our team members accountable for their assigned work and acted as an informal reference for tracking progress.
- Discord – in Fig. 4, members regularly update the team on their progress and status, specifically to notify the member responsible for the team's management and Gantt chart updates.

Code version control:

- Git – employed for version control of the game's codebase. Via Git, we systematically tracked changes, maintained a thorough commit history and aided in a three-member collaborative effort. Branching and merging conventions established clean integration of updated edits and minimised conflicts throughout the code inheritance.

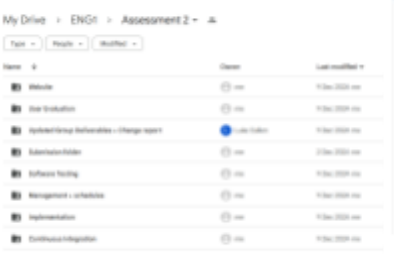


Fig. 1

Fig. 2

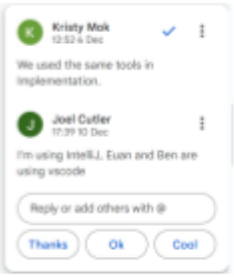


Fig. 3

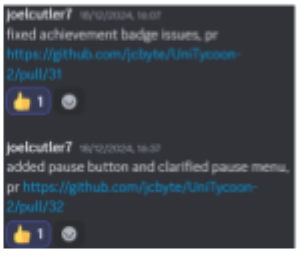


Fig. 4

b) i. Requirements

Old Requirements
New Requirements

In the description of the requirements, we extended the elicitation of requirements to include our second interaction with the client and detailed how we clarified the events and achievements, illustrated in Fig. 1. We felt it was necessary to keep this flow continuous so that the report did not contain any gaps in explanation for why things were done, and also allowed us to keep a clear log of the interactions that led to the requirements being sourced throughout the project, allowing for clear traceability from start to finish - something that is essential for ensuring that the end-result of the project is in alignment with what the client is looking for.

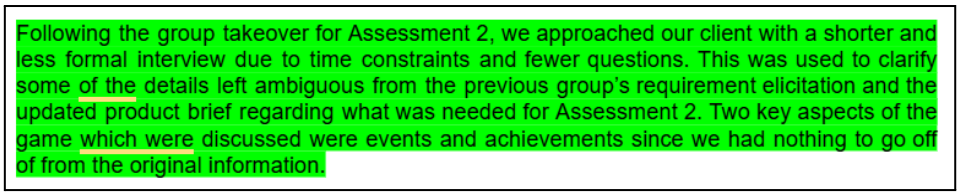
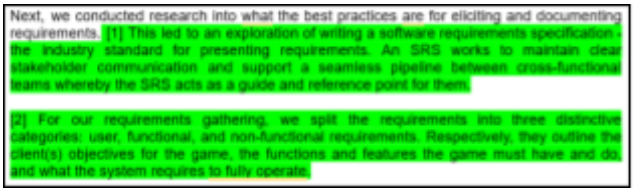

Following the group takeover for Assessment 2, we approached our client with a shorter and less formal interview due to time constraints and fewer questions. This was used to clarify some of the details left ambiguous from the previous group's requirement elicitation and the updated product brief regarding what was needed for Assessment 2. Two key aspects of the game which were discussed were events and achievements since we had nothing to go off of from the original information.

Fig. 1

One key aspect we noticed about the description for the requirements elicitation was that it had no references cited to justify the techniques used for eliciting and formatting the requirements in the way that they were done, so we researched and added the appropriate sources in Fig. 2a for the original approach they went for. It is important within the project that it is clear where information was sourced and properly explained, as shown in Fib. 2b, as to why things were done in the way they were done throughout the project.


Next, we conducted research into what the best practices are for eliciting and documenting requirements. [1] This led to an exploration of writing a software requirements specification - the industry standard for presenting requirements. An SRS works to maintain clear stakeholder communication and support a seamless pipeline between cross-functional teams whereby the SRS acts as a guide and reference point for them.

[2] For our requirements gathering, we split the requirements into three distinctive categories: user, functional, and non-functional requirements. Respectively, they outline the client(s) objectives for the game, the functions and features the game must have and do, and what the system requires to fully operate.

Fig. 2a


References:

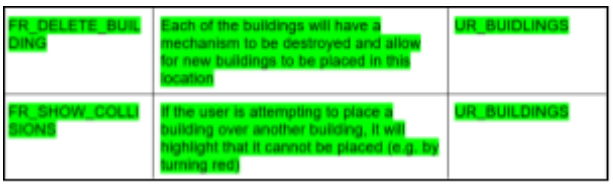
[1] GeeksforGeeks, "Software Requirement Specification (SRS) Format," GeeksforGeeks, Jun. 18, 2020. https://www.geeksforgeeks.org/software-requirement-specification-srs-format/

[2] "Importance of Requirement Gathering in Software Development," GeeksforGeeks, Jan. 30, 2024. https://www.geeksforgeeks.org/importance-of-requirement-gathering-in-software-developme nt/

Fig. 2b

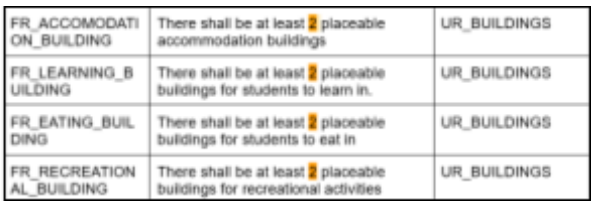
We decided to keep the structure of the requirements reference system's three tables: user requirements (URs), functional requirements (FRs), and non-functional requirements (NFRs). The advantage of keeping with the same system is that it means that the referencing remains consistent throughout the project and can be more easily tracked.

Analysing the previous group's requirements, we noticed some gaps in the requirements such as the buildings were too simplistic for what the client wanted, particularly with UR_BUIDLINGS. We therefore extended the FRs to match what was necessary for assessment 2, adding details about deletion of buildings and collision detection, shown in Fig. 3.



FR_DELETE_BUILDING	Each of the buildings will have a mechanism to be destroyed and allow for new buildings to be placed in this location	UR_BUIDLINGS
FR_SHOW_COLLISIONS	If the user is attempting to place a building over another building, it will highlight that it cannot be placed (e.g. by turning red)	UR_BUILDINGS

Fig. 3

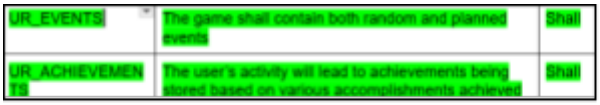


FR_ACCOMODATION_BUILDING	There shall be at least 2 placeable accommodation buildings	UR_BUILDINGS
FR_LEARNING_BUILDING	There shall be at least 2 placeable buildings for students to learn in.	UR_BUILDINGS
FR_EATING_BUILDING	There shall be at least 2 placeable buildings for students to eat in	UR_BUILDINGS
FR_RECREATIONAL_BUILDING	There shall be at least 2 placeable buildings for recreational activities	UR_BUILDINGS

Fig. 4

Some adjustments also had to be made to the pre-existing requirements in order to fit what was needed for this part of the assessment - we had to adjust the number of each building required from 1 to 2. Fig. 4 shows it highlighted in orange since this is a modification, rather than something that has been additionally added.

To the URs, we added the events and achievements UR_EVENTS and UR_ACHIEVEMENTS, as shown in Fig. 5, keeping in line with the new requirements from our client meeting. Some of the URs that were required for this stage of the assessment were already written by the previous team, such as the UR_LEADERBOARD, and so we didn't need to add as many as we would have otherwise needed to since these could be left unaltered. We aimed to match not only the type of requirements but also the style of naming conventions used - the previous group had opted for particularly simplistic and intuitive naming of requirements, such as the leaderboard being represented by "UR_LEADERBOARD". This is something that we aimed to imitate in the additional requirements, such as by adding achievements as "UR_ACHIEVEMENTS".



UR_EVENTS	The game shall contain both random and planned events	Shall
UR_ACHIEVEMENTS	The user's activity will lead to achievements being stored based on various accomplishments achieved	Shall

Fig. 5

FR_PLANNED_EVENTS	The system will include predictable, fixed events, such as graduation	UR_EVENTS
FR_UNPLANNED_EVENTS	The system will include random events such as weather conditions that cannot be predicted	UR_EVENTS
FR_MEASURE_ACHIEVEMENTS	The system will track user data in order to check whether certain goals have been achieved	UR_ACHIEVEMENTS
FR_DISPLAY_ACHIEVEMENTS	The achievements that the user has obtained will be clearly displayed to them	UR_ACHIEVEMENTS

Fig. 6

Then, for the FRs, we added the appropriate functionality needed to meet the URs, recognising the need often for multiple FRs to meet a single UR, as shown in Fig. 6.

As for the NFRs, we noticed that the client had expressed fundamentally wanting the same things out of the game now as they did at the beginning of the project, and so there was no need to change the NFRs at all, hence why the NFR table is identical. Since the previous group had been able to successfully meet the NFRs with what they had done so far, such as for NFR_COLOR, they had already implemented their buildings and background to match this requirement. This meant that instead of adding additional requirements here, the need was to extend the application of these NFRs, through things like our new building designs adhering to the NFR_COLOR requirement so that color-blind people would still be able to play the game successfully.

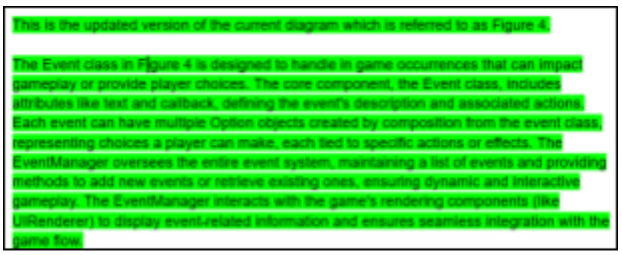
ii. Architecture

Old Architecture
New Architecture

The first issue we faced was working with an alternative tool - we had been previously familiar with Lucidchart but decided to adapt to using the other team's choice, PlantUML. This allowed us to be consistent with the previous group's designs and made it significantly easier to modify them for the adjustments necessary in Assessment 2.

When looking at the original architecture, code and requirements we realised that a few things were missing. There were no events in the original code so that was something the UML diagram needed to be updated with, as it was required from the product brief. Unlockable achievements and the leaderboard were also items omitted in their original diagram, which we later added to the final diagram.

No changes were made to the first 5 pages of the original document as it details their process of creating their architectural diagram. Instead, we modified their "Final Diagram" into "Current Diagram" as it was no longer the final one but rather a work-in-progress one. We then added two separate sections called the Updated Current and Final Diagrams detailing our evolution and continuation of their original architectural diagram into our final diagram depicted in Fig.2. A snippet of the evolution description is illustrated in Fig. 1.


This is the updated version of the current diagram which is referred to as Figure 4.

The Event class in Figure 4 is designed to handle in game occurrences that can impact gameplay or provide player choices. The core component, the Event class, includes attributes like text and callback, defining the event's description and associated actions. Each event can have multiple Option objects created by composition from the event class, representing choices a player can make, each tied to specific actions or effects. The EventManager oversees the entire event system, maintaining a list of events and providing methods to add new events or retrieve existing ones, ensuring dynamic and interactive gameplay. The EventManager interacts with the game's rendering components (like UIRenderer) to display event-related information and ensures seamless integration with the game flow.

Fig. 1.

By copying and modifying the existing PlantUML code, we created a new diagram to represent these changes; these are Figures 4 and 5 in the Arch2 document. We kept the original sections of the document, so we could still represent the evolution of the architecture. However, parts of the document were changed as their final diagram had been updated to accommodate our additions. This diagram contains the events, leaderboard and achievements in a section called util as they were the new user requirements, UR_EVENTS, UR_ACHIEVEMENTS and the original user requirement UR_LEADERBOARD. This was important as it clearly shows how the requirements were the foundations upon which the architecture was built.

Working on the implementation, these three new requirements were eventually put into their own folders to make the functionality of the code more modular. These were then implemented into the final class diagram of the code.

Figure 2: The final class diagram of Arch2

iii. Method Selection and Planning

Old Method Selection and Planning
New Method Selection and Planning

No changes were made to the programming methodology section because their analysis and arguments were sound and we adopted the same approach, the Agile methodology which we inherited from our Assessment 1 project lifecycle.

For the following collaboration and Development Tools, we modified and made additions to their existing pool of tools used throughout the whole project. We decided we did not need to add on any alternative considered as we were using the same tools we were using in Assessment 1, where we had already weighed and analysed the pros and cons of them. This spearheaded our progress and decision-making process when it came to adopting new/same tools for Assessment 2.

In Fig.1, we clearly differentiated what we used to document our notes in relation to what the original group used. We also added more detail into what we used within Google Suite, which was the use of Google Drive as this helped to clearly organise all of our deliverables.

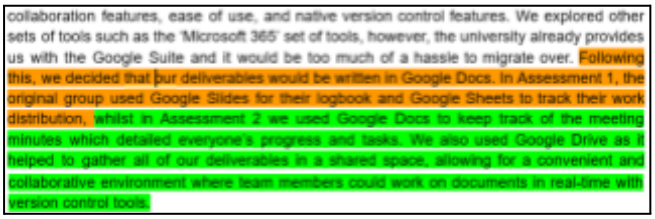
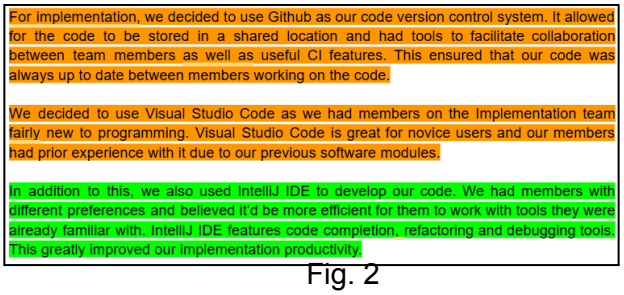

collaboration features, ease of use, and native version control features. We explored other sets of tools such as the 'Microsoft 365' set of tools, however, the university already provides us with the Google Suite and it would be too much of a hassle to migrate over. Following this, we decided that our deliverables would be written in Google Docs. In Assessment 1, the original group used Google Slides for their logbook and Google Sheets to track their work distribution, whilst in Assessment 2 we used Google Docs to keep track of the meeting minutes which detailed everyone's progress and tasks. We also used Google Drive as it helped to gather all of our deliverables in a shared space, allowing for a convenient and collaborative environment where team members could work on documents in real-time with version control tools.

Fig. 1


For implementation, we decided to use Github as our code version control system. It allowed for the code to be stored in a shared location and had tools to facilitate collaboration between team members as well as useful CI features. This ensured that our code was always up to date between members working on the code.

We decided to use Visual Studio Code as we had members on the Implementation team fairly new to programming. Visual Studio Code is great for novice users and our members had prior experience with it due to our previous software modules.

In addition to this, we also used IntelliJ IDE to develop our code. We had members with different preferences and believed it'd be more efficient for them to work with tools they were already familiar with. IntelliJ IDE features code completion, refactoring and debugging tools. This greatly improved our implementation productivity.

Fig. 2

As illustrated in Fig. 2, we elaborated more on the tools used in Implementation as the original group lacked analysis of the benefits and reason for using them. Our members also used IntelliJ IDE for this section, hence why it was listed along with its explanation for it.

To create our Gantt charts, we also used PlantUML, whereby we inherited the original group's script and built upon it. We also added an explanation in Fig. 3 as to why this was used and its fitness for our group dynamic as there was none previously. In addition, we used Canva for our structural diagrams (work breakdown), which the original group had not used.

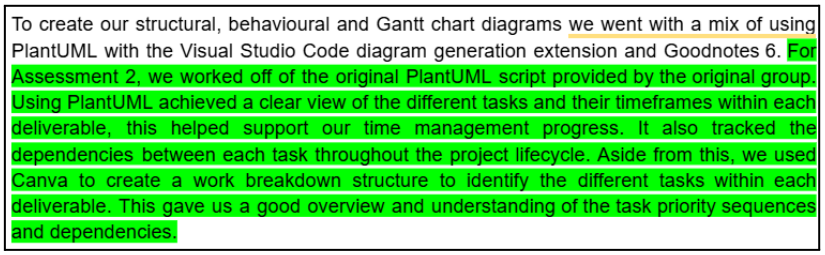

To create our structural, behavioural and Gantt chart diagrams we went with a mix of using PlantUML with the Visual Studio Code diagram generation extension and Goodnotes 6. For Assessment 2, we worked off of the original PlantUML script provided by the original group. Using PlantUML achieved a clear view of the different tasks and their timeframes within each deliverable, this helped support our time management progress. It also tracked the dependencies between each task throughout the project lifecycle. Aside from this, we used Canva to create a work breakdown structure to identify the different tasks within each deliverable. This gave us a good overview and understanding of the task priority sequences and dependencies.

Fig. 3

After a thorough discussion, we decided it was best to use generative AI, DALL-E, to generate our new sprites to adhere to the new product brief. This was because our members had not used Aseprite and Photoshop prior to this and believed that DALL-E was more time-efficient and reliable in terms of producing sprites with a consistent art style to the originals. We added this decision to the artwork section in Fig. 4.

For the artwork, we used a mixture of Aseprite and Photoshop to create the UI and sprites owing to their versatility and plethora of features. To maintain a seamless and consistent display of art style from the original sprites, we utilised DALL-E to create the new features following Assessment 2. Using generative AI, we fed it screenshot examples of the original sprites and gave it detailed descriptions of what results we wanted to achieve, specifying that we wanted it to follow the same art style input.

Fig. 4

We modified their communication tools paragraph as shown in Fig. 5 since they had previously compared Discord (with advantages written down) and WhatsApp, and they ultimately decided to go with the latter. However, we used Discord throughout our project, hence we incorporated the two communication platforms and rearranged both theirs and our reasoning for their usage.

As for communication channels, we chose WhatsApp (original group) and Discord (Assessment 2 group) to be our main points of contact. Discord allowed for the transferring of larger files and the ability to be able to share your screen during team calls, whilst WhatsApp was already installed on our members' phones which meant they would be quick to respond to any important messages if needed, which is something very important within the agile methodology; response and engagement.

Fig. 5

Their team organisation section briefly touched upon how they allocated each member's role and took into account the mark distribution. However, they did not consider how their approaches were appropriate for the project as a whole. Therefore, whilst adding our team organisation process as shown in Fig. 6, we made sure to address additional factors such as low bus factor and having one leader for each deliverable. They also mentioned that they updated their logbook twice a week during their meetings and explained how they did so. We followed a similar approach so we did not change this area.

From the beginning of Assessment 2, we discussed and negotiated our respective roles and responsibilities. Following on from our previous roles in Assessment 1, we concluded that we would stick to the same roles in Assessment 2 with a few minor changes. This was because we felt comfortable and more knowledgeable in our respective areas, allowing us to work more efficiently and effectively. Due to the mark contribution for different sections, we also negotiated who should take up the newly added deliverables to ensure everyone had on average similar mark contributions.

Throughout the allocation of new roles, we made sure to consider the low bus factor through the number of people working on the same deliverable. This was to support better risk mitigation across the project lifecycle. An example of this is our Implementation and Software Testing teams, whereby we decided on keeping the same three people for the two deliverables as these sections weighed heavier than most and that the marks would be better distributed this way. This also allowed for a more seamless and consistent style and quality as the two sections are closely intertwined, with iterative checks between them.

Each deliverable had its respective single leader to oversee and manage its tasks. Doing so supports a clearer vision/overview of the deliverable and reduces any confusion about each member's work.

Fig. 6

For the systematic plan for the project, we added an entirely separate section from theirs as we were working on Assessment 2 which had a different overview of tasks compared to Assessment 1, but made sure to follow the same format as the original. Furthermore, we added a work breakdown structure as illustrated in Fig. 7 to clearly outline the tasks and their priorities/dependencies within each deliverable section. The original group did not create one, but rather created an initial Gantt chart to outline their project and made mini-weekly ones to show their weekly progress briefly. We did not opt for this approach but instead made iterations and changes to our overall Gantt chart every week during our progress check-ins. Our Gantt chart was written in more detail to support better referencing and clearer sequence progress which helps to guide our team members on their tasks.

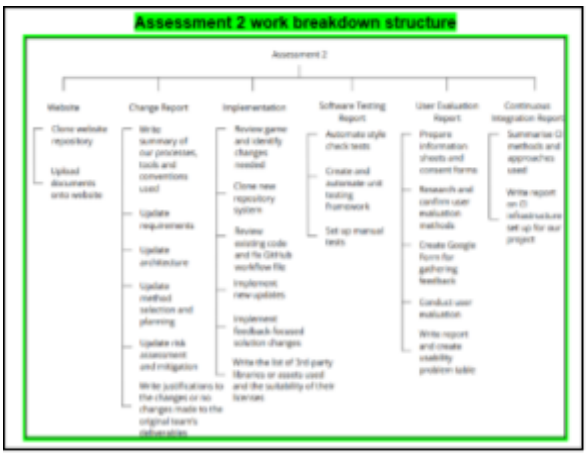


Fig. 7

Fig. 8

After inheriting the original PlantUML from their Assessment 1, in Fig. 8, we decided to change the colour palette of the Gantt chart section which we would later build upon from the original to clearly differentiate the two phases.

The example shown in Fig. 9 is a snippet from the rest where we discuss how our plan evolved throughout Assessment 2 and Fig. 10 shows the section pointing out our weekly snapshots on our website.

Week 14: 30th Dec - 3rd Jan

Our team's progress slowed down significantly due to a lack of communication and slow responses from members as it was still the holidays. This meant that we started on the latter sections of Assessment 2 later than we'd hoped.

Throughout the implementation process, our team consistently practised continuous integration, whereby our members updated and shared their progress almost daily and multiple times a day. This week, we started writing the first part of the Continuous Integration report.

We started on software testing, where we added style check testing and automated it. Afterwards, we manually went through the entire project refactoring it to pass all the style tests. The framework for unit testing was created and also automated during this week. We had three members working on software testing as a part of the implementation section, hence there was a handover, resulting in a delay in the workflow for this area.

Fig. 9

Weekly snapshots of how our project plan evolved are uploaded on our website: https://vikingz-updated.vercel.app/pdfs/timeline.pdf

Fig. 10

IV) Risk Assessment and Mitigation

Old Risk Assessment
Updated Risk Assessment

The Risk1 document has been updated in Risk2 to include new risks in the risk register, along with updating already existing risks to include our group where applicable. The risks that have been added to the risk register are; R9, R10, R11, and R12, illustrated in Fig. 1.

These risk IDs are highlighted in green to easily identify the new risks that are posed to our team upon taking up another project. These risks arise mostly because we have to adapt to work that has been worked on by other people, which introduces new problems that weren't possible before. These risks have been identified in order for our team to meet the requirements of Assessment 2, with mitigations in place to ensure that we can avoid risks that are identified, or take appropriate action when any of these risks have been breached.

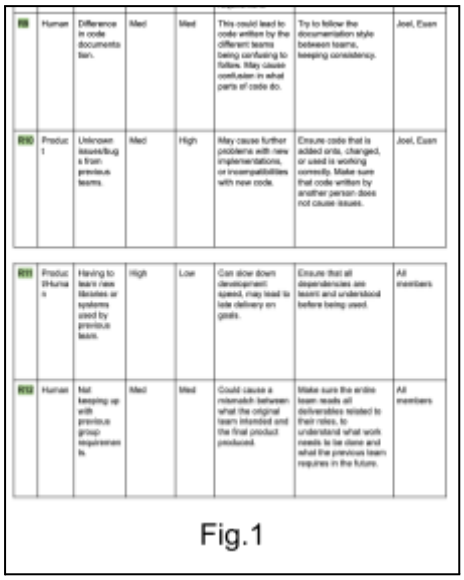


Fig.1

These risk identifications are important as they may have gone unnoticed without being addressed, along with their potential harm to our product, timelines and workflow being large. For example, slowing down development speed may lead to delayed timelines, and not reaching our goals on our set times. This may also impact further needs, such as user testing if the product is not to a necessary standard within our set times.

Any risks that were identified by the previous team, which needed to be updated or changed had these updates highlighted in Orange, as shown in Fig. 2. These were risks that were either still relevant to our team or needed to be changed to fit in with our new role and requirements for the product.

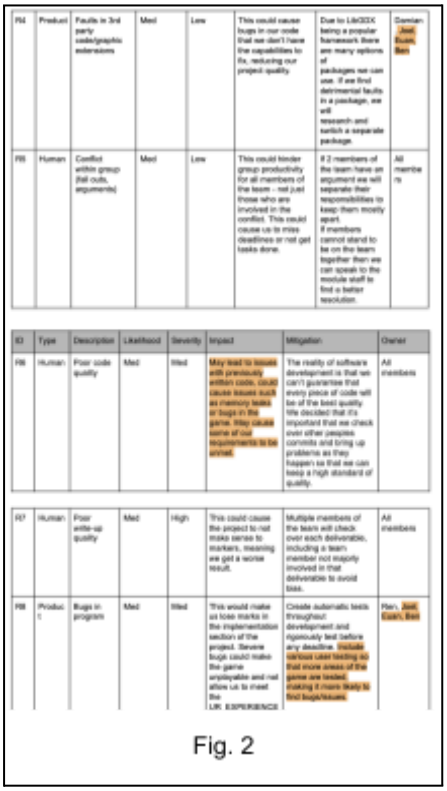


Fig. 2

Most of the risks that needed to be changed involved including members of our team in the risks already identified. As those risks are still relevant in our roles with the mitigations and impact being similar. Some changes, however, needed to be made to the impact or mitigation of previous risks. This is due to our roles now having different requirements and responsibilities, which altered the way the risks should be managed or may affect our team.

These changes are also important as they allow us to use the previously identified risks, without missing risks that appear from our new roles with similar tasks. It prevents us from causing time delays or missing deliverables while ensuring we can manage any previous risks correctly without causing any further delays or problems to product development.

Ensuring that we anticipate risks and changes to these risks is also important for team morale, as unforeseen

problems are likely to decrease our motivation for working towards our goals. This makes it important that the risk register is updated, to be confident that our goals are realistic and reachable, while the team can be driven to achieve them.

Collaboration between our team and the original team was also an important factor to consider when the new risks and changes were identified, as it was likely that our styles of working would be different. This meant that going forward with building upon their original work, we had to ensure our documentation quality was clear and that detailed planning would be crucial. The layout of the other team's risk register did not need to be changed, however we could continue using it in the same fashion, due to it being simple and well laid out. With it being clearly shown and identified, the severity and likelihood of risks occurring allow our team to predict where risks may occur without being overly cautious.